

Tema 8: Punteros y memoria dinámica

1. La memoria en C++: Stack vs Heap

Para entender los punteros, primero debemos saber dónde se almacenan los datos. La memoria de un programa se divide principalmente en dos zonas:

Característica	Stack (Pila)	Heap (Montículo)
Gestión	Automática (por el ordenador).	Manual (por el programador).
Velocidad	Muy rápida.	Más lenta.
Tamaño	Limitada y fija.	Muy grande (memoria RAM disponible).
Ciclo de vida	Ligado a las llaves {} (stack frames).	Decidimos cuándo nacen (new) y mueren (delete).

Ejemplo de scope (Pila):

```
int f(int a, int b) {  
    int n = a;  
    if (b > n) {  
        int m = 2; // Nace aquí  
        a = b;  
    } // m muere aquí automáticamente  
    return a;  
} // n y a mueren aquí
```

2. ¿Qué es un puntero?

Un **puntero** es una variable que, en lugar de almacenar un valor (como un `int` o `char`), almacena una **dirección de memoria**.

Operador	Nombre	Función	Ejemplo
<code>&</code>	Dirección de	Obtiene la dirección de memoria de una variable.	<code>p = &x;</code>
<code>*</code>	Desreferencia	Accede al contenido de la dirección que guarda el puntero.	<code>cout << *p;</code>
<code>-></code>	Flecha	Acceso a miembro vía puntero. Equivalente a <code>(*p).miembro</code> .	<code>pp->first = "b";</code>

Trampas de declaración y sintaxis

- **Declaración múltiple:** El asterisco `*` debe ponerse por cada variable.

```
int *pb, *pc; // Dos punteros
int* pb, pc; // ¡pb es puntero, pc es un ENTERO normal! (Error típico)
```

- Puntero a elementos de contenedores:

```
vector<int> v = {1, 2, 3};
int *p = &v[1]; // Apunta al '2'
*p += 1;        // v[1] ahora es 3
```

- Puntero a miembros de pair o struct:

```
pair<string, int> a = {"a", 7};
int *pi = &a.second;
*pi = 0; // a.second ahora es 0
```

nullptr vs Inicialización: - `int *p = nullptr;` -> El puntero apunta a “nada”. Seguro. - `int *p;` -> El puntero apunta a una **dirección aleatoria** (basura). Muy peligroso. - `int *p = 5;` -> **ERROR**. Estás diciendo que la dirección de memoria es la número 5. Esto provocará un **SEGFAULT** seguro.

```
int x = 10;
int* p = &x; // p apunta a x

cout << p;    // Imprime una dirección: 0x7ffe...
cout << *p;   // Imprime el valor de x: 10
```

3. Gestión dinámica de memoria

Esta es la utilidad real de los punteros: pedir memoria en tiempo de ejecución.

Objetos vs vectores dinámicos

Podemos pedir un solo objeto o un bloque entero (vector) al Heap usando **new**, y liberarlo con **delete**.

```
// Objetos simples
Data *pd = new Data(2025, 4, 2);
pair<int, int> *pp = new pair<int, int>(1, 2);
```

```
// Vectores dinámicos (Muy común en C)
int *pv = new int[100];
char *pc = new char[100000];
```

Memory Leak: Se produce cuando pierdes la dirección y ya no puedes hacer **delete**.

```
int *p = new int[100];
p = new int[100]; // ERROR: ¡Se ha perdido la dirección del primer vector! Fuga de memoria.
```

4. Aliasing y asignación

El **aliasing** ocurre cuando dos o más punteros apuntan a la misma dirección de memoria. Modificar el valor a través de un puntero afecta a todos los otros “alias”.

```
int x = 10;
int* p1 = &x;
int* p2 = p1; // Aliasing: p2 apunta donde apunta p1

*p2 = 20;
cout << x; // ¡Imprimirá 20!
```

Ejemplo avanzado: Un vector de punteros apuntando al mismo objeto.

```
int x = 3;
vector<int*> v(10, &x); // 10 punteros que apuntan TODOS a la x

for (int i = 0; i < v.size(); ++i) {
    (*v[i])++; // ;Incrementamos x 10 veces!
}
cout << x; // Imprimirá 13
```

5. El peligro de los punteros: errores comunes

El uso de punteros requiere mucha disciplina. Los errores más habituales en PRO2 son:

1. **Segmentation Fault (SEGFAULT)**: Intentar acceder a una dirección que no te pertenece.
 - Desreferenciar nullptr: `int *p = nullptr; *p = 5;`
 - Acceso fuera de rango en vectores: `vector<int> v; v[13] = 0;`
2. **Memory Leak**: Destruir la única referencia a una memoria dinámica sin liberarla.
3. **Dangling Pointer (Puntero colgante)**: Puntero que apunta a una dirección que ya ha sido liberada con `delete`.
4. **Double-Delete**: Hacer `delete` dos veces sobre la misma dirección (corrompe el heap).

En el Makefile, utiliza el flag `-D_GLIBCXX_DEBUG`. Esto activa comprobaciones de seguridad en los contenedores de la STL y te avisará de los accesos fuera de rango en lugar de darte un SEGFAULT silencioso o datos basura.

Operación	Iterador (STL)	Puntero (Bajo Nivel)
Inicio	<code>auto it = v.begin();</code>	<code>int *px = &x;</code>
Acceso	<code>*it = 5;</code>	<code>*px = 5;</code>
Avanzar	<code>it++;</code>	<code>px++;</code> (Avanza una dirección)
Reasignar	<code>it = v.erase(it);</code>	<code>px = &y;</code>

Un puntero puede ser visto como un iterador de un vector, pero un iterador de un `std::list` no es necesariamente un puntero (internamente puede ser más complejo).

6. Paso de parámetros

Tipo	Sintaxis	Efecto	Eficiencia
Por valor	<code>f(int x)</code>	Copia del valor.	Baja (si el objeto es grande).
Por referencia	<code>f(int& x)</code>	Alias directo.	Alta.
Por puntero	<code>f(int* pi)</code>	Pasa la dirección. Más rápido que por valor.	

Ejemplo de incremento:

```
void inc(int *pi) {
    (*pi)++; // quiere decir *pi += 1;
}

int i = 5;
inc(&i); // i ahora vale 6
```

En PRO2, preferimos **referencias constantes** (`const T& x`) para objetos grandes que no queremos modificar, y **punteros** solo cuando necesitamos que el parámetro pueda ser opcional (`nullptr`) o para estructuras dinámicas.

7. Aplicación: Estructuras Enlazadas (Nodos)

La utilidad real de los punteros en PRO2 es crear colecciones de datos que crecen y decrecen nodo a nodo. Cada elemento se almacena en un **Nodo** (o **Item**).

El Nodo Básico

```
struct Item {
    T value;      // El dato
    Item* next;  // El "cable" hacia el siguiente nodo
};
```

7.1 La Pila (Stack) - Modelo LIFO

En una pila enlazada, solo tenemos un puntero a la cima (`ptopitem`). Cada vez que hacemos **push**, el nuevo nodo apunta a la antigua cima.

::stackviz

[!TIP] **Ejercicio `swap2Topmost()`**: Para intercambiar los dos primeros nodos sin tocar los `.value`, debes: 1. Guardar el segundo nodo en un puntero auxiliar: `Item *p2 = ptopitem->next`; 2. Reenlazar: `ptopitem->next = p2->next`; 3. Hacer que el nuevo primero sea el que era segundo: `p2->next = ptopitem`; 4. Actualizar la cima: `ptopitem = p2`;

7.2 La Cola (Queue) - Modelo FIFO

En una cola, necesitamos dos punteros: **first** (para sacar) y **last** (para añadir).

::queueviz

[!IMPORTANT] **Ejercicio `operator[]`**: Como una cola enlazada no es un vector, para encontrar el elemento `i` tienes que hacer un bucle `for` que avance el puntero `p = p->next` exactamente `i` veces empezando desde `first`.

7.3 Cómo borrar nodos por el medio

Para borrar un nodo (como en `removeFirstOccurrence`), necesitas “saltártelo”: 1. Encontrar el nodo **anterior** al que quieres borrar (`ant`). 2. Hacer el salto: `ant->next = ant->next->next`; 3. Liberar la memoria: `delete p_a_borrar`;

::pointerviz

Checklist para el Juez (Punteros)

- ☐ **¿Has puesto `nullptr`**? Comprueba siempre si un puntero es nulo antes de hacer `p->next`.
- ☐ **¿Has hecho `delete`**? Cada `new` debe tener su `delete` para evitar Memory Leaks.
- ☐ **Casos vacíos**: ¿Qué hace tu código si la pila/cola está vacía? ¿Y si tiene solo 1 elemento?
- ☐ **Auto-asignación**: En el uso de `operator=`, ¿has comprobado `if (this != &s)?`